Session 7

AADK Task 7

Assessment Report on State Concepts Deep Dive & Visual Mapping , Input, Validation & UI Feedback Plan , Comprehensive Feature Design & Implementation Strategy

Name : Pawankumar

Course : Android App Development with Gen AI

These three tasks for students learning Jetpack Compose in Android development. The focus is on state management, user input validation, and comprehensive feature design. Each task is explained in depth, including key concepts, practical action items, submissions, and outcomes. Relevant images, flow diagrams, and block diagrams are included with explanations. The content is structured for clarity and can be copied into a tool like Microsoft Word or Google Docs to export as PDF.

Jetpack Compose

in Jetpack Compose, showing how activities and processes interact in the Android lifecycle. It highlights methods like onCreate(), onStart(), etc., which are crucial for understanding how state persists or recomposes in Compose UIs.

Task 1

State Concepts Deep Dive & Visual Mapping

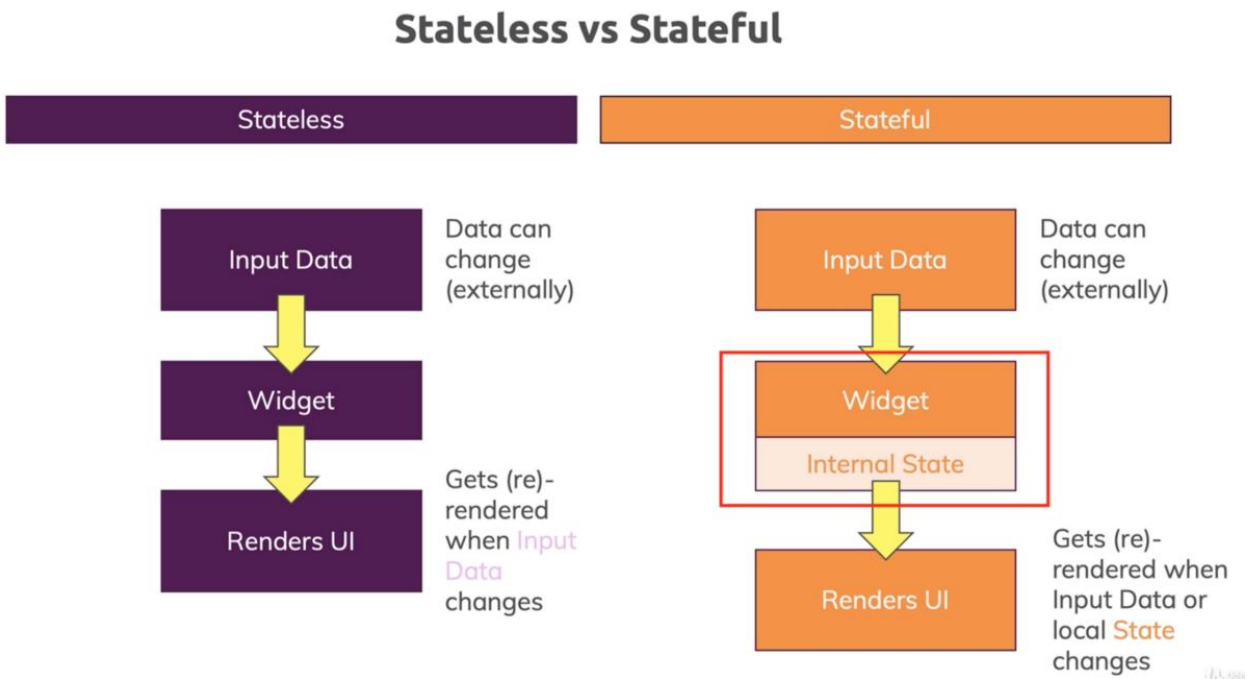
This task requires students to explore the core concept of "state" in Jetpack Compose. State refers to data that can change over time and triggers UI updates (recomposition). Key elements include:

- **State**: Mutable data that drives UI rendering. In Compose, state changes cause the UI to recompose efficiently.
- **remember()**: A function to retain state across recompositions, preventing reset on UI redraws.
- **mutableStateOf()**: Creates a mutable state holder, often used with remember() for reactive variables.
- **State Hoisting**: Moving state to a higher-level composable for better management, reusability, and testing.
- **Recomposition**: The process where Compose re-runs composable functions when state changes, updating only affected UI parts.

Students must document definitions, provide pseudo-code examples, select a scenario (e.g., tip calculator), create flow diagrams, build a state-handling table, and reflect on state management.

Practical Action Items:

1. Research & Document Key Terms: Define terms and give examples, e.g., pseudo-code for mutableStateOf: val count = remember { mutableStateOf(0) }.
2. Select an Interaction Scenario: e.g., User enters bill → Tip calculated → Toggle rounds tip → Display result.
3. State-Handling Strategy Table: Columns include UI Component, State Variable, Stateful/Stateless, Definition Location, Comments.
4. Reflection: Discuss why hoisting prevents bugs like inconsistent UI updates.



Explanation of Block Diagram: This block diagram compares stateless vs. stateful composables. Stateless ones rely on external input, while stateful manage internal state. It visualizes data flow:

Input → Widget → Renders UI, emphasizing how internal state enables independent updates.

Custom Flow Diagram for Tip Calculator Scenario

Yes

No

User Input: Enter Bill Amount

Update billAmount State

Calculate Tip: tip = billAmount * tipPercentage

Toggle Round Up?

Round Tip Up

Keep Original Tip

Update result State

Recompose UI: Display Result

This diagram maps user input to state changes. Starting from input, it updates state, applies logic (e.g., rounding), and triggers recomposition. Arrows show transitions, with decisions for toggles.

Task 2

Input, Validation & UI Feedback Plan

Building on Task 1, this focuses on interactive features with inputs (TextFields), validation, switches, and feedback. Validation ensures data integrity (e.g., bill > 0), while feedback provides UX cues like error messages or color changes.

Practical Action Items:

1. Define Feature Scenario: e.g., Expense Splitter – Enter bill, people count, round-up toggle; displays per-person amount.
2. Input & State Variables: List fields (e.g., billAmountInput: String, validation: >0; numPeopleInput: Int, >=1).
3. UI Feedback: For invalid input, show red border/error text; disable buttons; on success, update result.
4. Wireframe & Flow Sketch: Sketch screen with labels for composables (TextField, Switch, Button) and modifiers.
5. Reflection: Importance of validation for UX and preventing crashes.

Bill Splitting App Concept Sketch freebie - Download free resource for Sketch - Sketch App Sources

Explanation of Wireframe Diagram: This wireframe shows a bill splitting app UI with screens for owing amounts, adding bills, and history. It includes inputs, buttons, and feedback zones like paid-by labels.

Case study: Designing a bill splitting app | by Swati Raj | Bootcamp | Medium

Explanation of Wireframe Diagram: Compares old vs. redesigned edit expense screen, highlighting flexible member addition and photo upload for validation proof.

Full Guide: How to Form Validation With Jetpack Compose | by Meet | ProAndroidDev

Explanation of Image: Demonstrates form validation in Compose, with error messages like "Password must be 8 to 10 Characters!" under fields, showing real-time feedback.

Custom Flow Diagram for Validation

Valid

Invalid

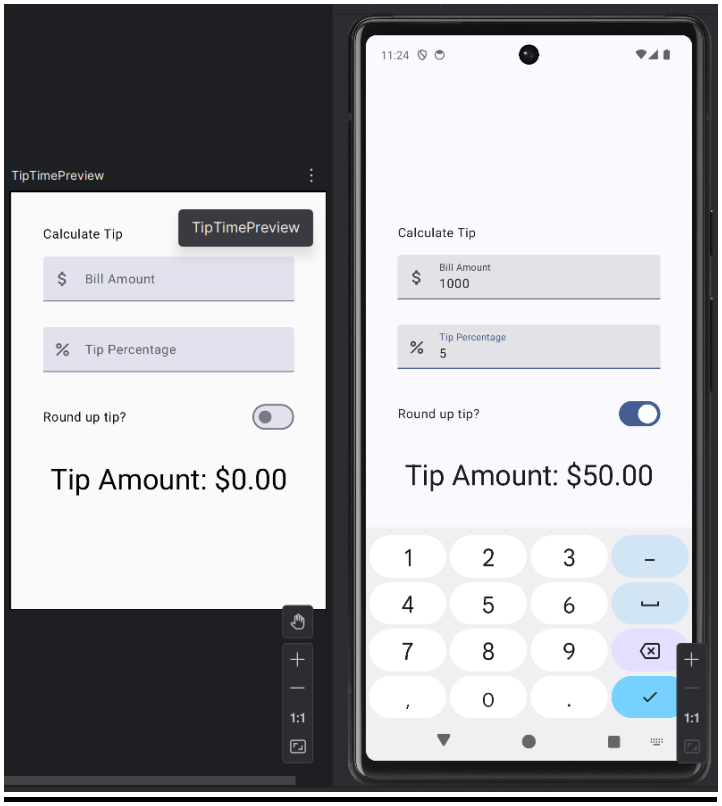
User Enters Input

Validate Rules e.g., >0?

Update State & Enable Button

Show Error Message & Red Border

Calculate & Display Result



Task 3

Comprehensive Feature Design & Implementation Strategy

Detailed Explanation

This capstone task integrates everything into a complex feature like "Bill Split & Tip Tracker". It includes multiple inputs, derived states, history saving, testing, and scalability.

Practical Action Items:

1. Define Feature: Summary of purpose (e.g., split bills with tips, save history).
2. Implementation Strategy: Layout sketch, button descriptions (e.g., "Save" updates historyList), state model table (e.g., billAmount: mutable, initial=0; result: derived).
3. Pseudo-code: e.g., historyList.add(calculatedSplit).
4. Testing & Edge-Cases: Valid/invalid/large inputs; UI tests for orientation.
5. Scaling: Extend to currencies or persistent storage.
6. Reflection: Complexity in state management, gains in Compose skills.

Custom Block Diagram for State Model (Text-Based)

Passes State

BillSplitViewModel

+billAmount: MutableState

+numPeople: MutableState

+tipPercentage: MutableState

+roundUp: MutableState

+historyList: MutableState>

+result: State(derived)

UIComponents

Code of Tip Calculator :

```
package com.example.tipcalculator

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.annotation.DrawableRes
import androidx.annotation.StringRes
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.tipcalculator.ui.theme.TipCalculatorTheme
import java.text.NumberFormat
import kotlin.math.ceil

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()

        setContent {
            TipCalculatorTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    TipTimeLayout()
                }
            }
        }
    }
}

@Composable
```

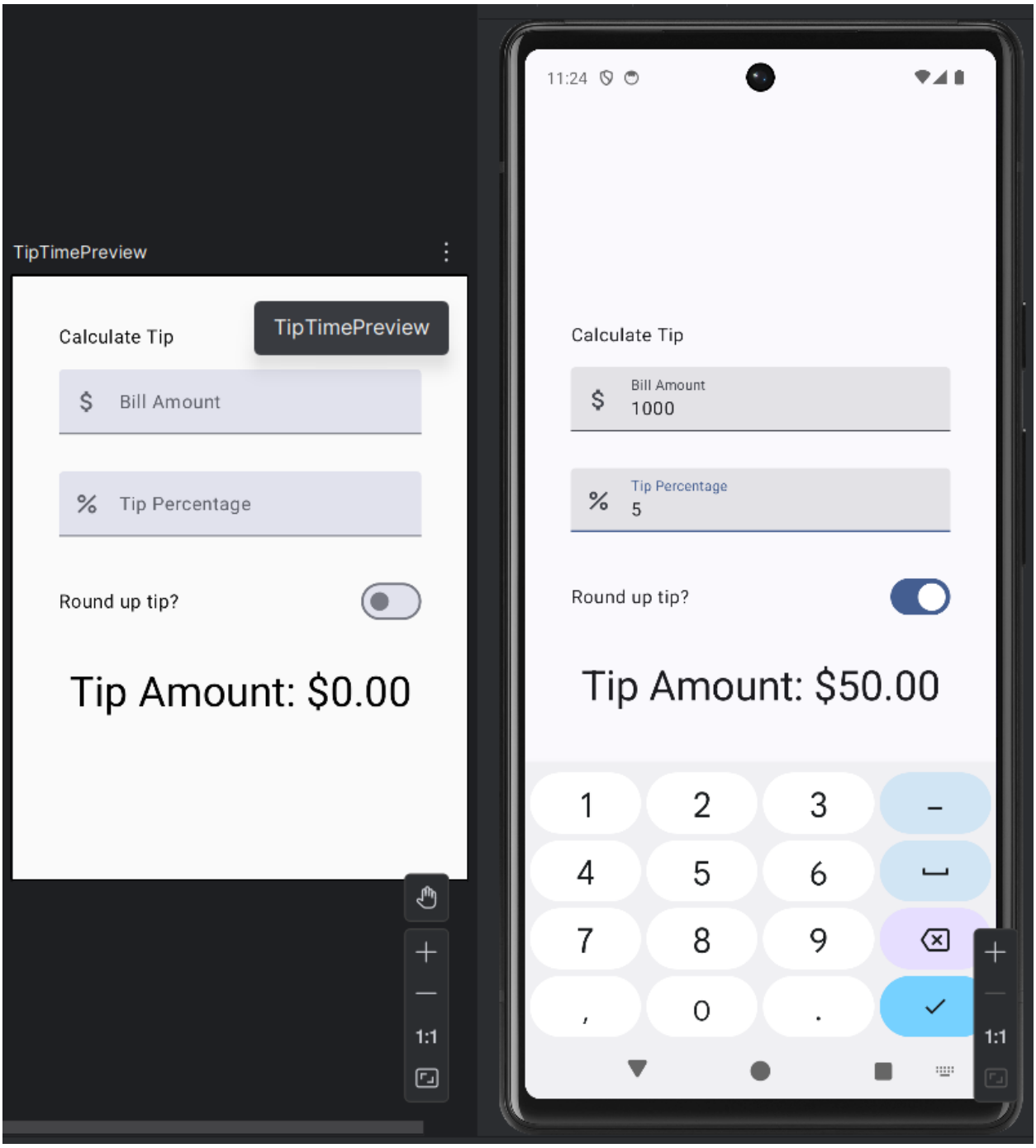
```
fun TipTimeLayout() {

    var amountInput by remember { mutableStateOf("") }
    var tipInput by remember { mutableStateOf("") }
    var roundUp by remember { mutableStateOf(false) }

    val amount = amountInput.toDoubleOrNull() ?: 0.0
    val tipPercent = tipInput.toDoubleOrNull() ?: 0.0

    val tip = calculateTip(amount, tipPercent, roundUp)

    Column(
        modifier = Modifier
            .padding(40.dp)
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {

        Text(
            text = stringResource(R.string.calculate_tip),
            modifier = Modifier
                .padding(bottom = 16.dp)
                .align(Alignment.Start)
        )

        EditNumberField(
            label = R.string.bill_amount,
            leadingIcon = R.drawable.money,
            keyboardOptions = KeyboardOptions.Default.copy(
                keyboardType = KeyboardType.Number,
                imeAction = ImeAction.Next
            ),
            value = amountInput,
            onValueChanged = { amountInput = it },
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 32.dp)
        )

        EditNumberField(
            label = R.string.how_was_the_service,
            leadingIcon = R.drawable.percent,
            keyboardOptions = KeyboardOptions.Default.copy(
                keyboardType = KeyboardType.Number,
                imeAction = ImeAction.Done
            ),
            value = tipInput,
```

```
            onValueChanged = { tipInput = it },
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 32.dp)
        )

        RoundTheTipRow(
            roundUp = roundUp,
            onRoundUpChanged = { roundUp = it },
            modifier = Modifier.padding(bottom = 32.dp)
        )

        Text(
            text = stringResource(R.string.tip_amount, tip),
            style = MaterialTheme.typography.displaySmall
        )

        Spacer(modifier = Modifier.height(100.dp))
    }
}

@Composable
fun EditNumberField(
    @StringRes label: Int,
    @DrawableRes leadingIcon: Int,
    keyboardOptions: KeyboardOptions,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
) {

    TextField(
        value = value,
        onValueChange = onValueChanged,
        label = { Text(stringResource(label)) },

        leadingIcon = {
            Icon(
                painter = painterResource(id = leadingIcon),
                contentDescription = null
            )
        },

        singleLine = true,
        keyboardOptions = keyboardOptions,
        modifier = modifier
    )
}
```

```
@Composable
fun RoundTheTipRow(
    roundUp: Boolean,
    onRoundUpChanged: (Boolean) -> Unit,
    modifier: Modifier = Modifier
) {

    Row(
        modifier = modifier
            .fillMaxWidth()
            .size(48.dp),

        verticalAlignment = Alignment.CenterVertically
    ) {

        Text(text = stringResource(R.string.round_up_tip))

        Switch(
            checked = roundUp,
            onCheckedChange = onRoundUpChanged,
            modifier = Modifier
                .fillMaxWidth()
                .wrapContentWidth(Alignment.End)
        )
    }
}

private fun calculateTip(
    amount: Double,
    tipPercent: Double,
    roundUp: Boolean
): String {

    var tip = tipPercent / 100 * amount

    if (roundUp) {
        tip = ceil(tip)
    }

    return NumberFormat.getCurrencyInstance().format(tip)
}

@Preview(showBackground = true)
@Composable
fun TipTimePreview() {
    TipCalculatorTheme {
        TipTimeLayout()
```

```
    }
}
```

Output :


TipTimePreview
Calculate Tip
TipTimePreview
Bill Amount
Tip Percentage
Round up tip?
Tip Amount: $0.00
1:1
11:24
Calculate Tip
Bill Amount
1000
Tip Percentage
5
Round up tip?
Tip Amount: $50.00
1:1